

TOOLS

HelixCode Tools Documentation

Overview

HelixCode provides a comprehensive tool ecosystem for AI-powered development workflows. The tools are organized into categories and accessible through a unified registry that provides schema validation, type safety, and automatic resource management.

Architecture

Tool Registry

The ToolRegistry is the central hub for all tools. It:

- Manages tool lifecycle and dependencies
- Provides schema validation for parameters
- Handles resource cleanup and error recovery
- Supports tool aliases for convenience
- Exports OpenAPI-compatible schemas

```
registry, err :=
    tools.NewToolRegistry(tools.DefaultRegistryConfig())
defer registry.Close()

// Execute a tool
result, err := registry.Execute(ctx, "fs_read",
    map[string]interface{}{
        "path": "/path/to/file.go",
    })
```

Tool Categories

1. **FileSystem** - File operations (read, write, edit, search)
2. **Shell** - Command execution (sync, async, background)
3. **Web** - HTTP requests and web scraping
4. **Browser** - Browser automation and screenshots
5. **Mapping** - Codebase analysis and symbol extraction
6. **MultiEdit** - Transactional multi-file editing

- 7. **Interactive** - User prompts and task tracking
- 8. **Notebook** - Jupyter notebook manipulation

Tool Reference

FileSystem Tools

fs_read

Read file contents from the filesystem.

Parameters: - path (string, required): Path to the file to read - start_line (integer, optional): Start line number for partial read - end_line (integer, optional): End line number for partial read

Returns: FileContent object with content, lines, metadata

Example:

```
content, err := registry.Execute(ctx, "fs_read",  
    map[string]interface{}{  
        "path": "main.go",  
    })
```

Security: Validates paths against workspace boundaries, blocks access to sensitive files (.env, credentials, etc.)

fs_write

Write content to a file atomically.

Parameters: - path (string, required): Path to the file to write - content (string, required): Content to write - backup (boolean, optional): Create backup before writing

Returns: nil on success

Example:

```
err := registry.Execute(ctx, "fs_write", map[string]interface{}{  
    "path": "output.txt",  
    "content": "Hello, World!",  
    "backup": true,  
})
```

Security: Requires write permissions, creates backups, uses atomic writes to prevent corruption

fs_edit

Edit file contents by replacing strings.

Parameters: - path (string, required): Path to the file to edit - old_string (string, required): String to replace (must be unique unless replace_all is true) - new_string (string, required): Replacement string - replace_all (boolean, optional): Replace all occurrences

Returns: EditResult with diff and statistics

Example:

```
result, err := registry.Execute(ctx, "fs_edit",
    map[string]interface{}{
        "path": "config.go",
        "old_string": "localhost:8080",
        "new_string": "0.0.0.0:8080",
    })
```

Best Practices: - Ensure old_string is unique in the file to avoid ambiguous replacements - Use replace_all: true for renaming variables/identifiers - Review the diff before applying changes

glob

Find files matching a glob pattern.

Parameters: - pattern (string, required): Glob pattern (e.g., **/*.go, src/**/*.ts) - root (string, optional): Root directory to search from

Returns: Array of matching file paths

Example:

```
files, err := registry.Execute(ctx, "glob",
    map[string]interface{}{
        "pattern": "**/*.go",
    })
```

Supported Patterns: - * - matches any sequence of characters - ? - matches any single character - ** - matches any number of directories - [abc] - matches any character in the set - {a,b} - matches either pattern

grep

Search file contents for a pattern.

Parameters: - pattern (string, required): Pattern to search for - root (string, optional): Root directory to search from - regex (boolean, optional): Use regex pattern matching - case_sensitive (boolean, optional): Case sensitive search (default: true) - max_matches (integer, optional): Maximum number of matches to return

Returns: Array of ContentMatch objects with file path, line number, and context

Example:

```
matches, err := registry.Execute(ctx, "grep",
    map[string]interface{}{
        "pattern": "TODO:",
        "regex": false,
        "case_sensitive": false,
    })
```

Performance Tips: - Use max_matches to limit results for large codebases - Exclude directories like node_modules in configuration - Use regex for complex patterns but be aware of performance impact

Shell Tools

shell

Execute a shell command synchronously.

Parameters: - command (string, required): Shell command to execute - workdir (string, optional): Working directory - timeout (integer, optional): Timeout in seconds (default: 30) - env (object, optional): Environment variables

Returns: ExecutionResult with stdout, stderr, exit code

Example:

```
result, err := registry.Execute(ctx, "shell",
    map[string]interface{}{
        "command": "go test ./...",
        "timeout": 300,
    })
```

Security: - Commands are validated against blocklist (rm -rf /, dd, mkfs, etc.) -
Runs in sandbox with resource limits - Audit logging for dangerous commands -
Environment isolation

shell_background

Execute a shell command asynchronously in the background.

Parameters: - command (string, required): Shell command to execute - workdir (string, optional): Working directory - env (object, optional): Environment variables

Returns: AsyncExecution object with execution ID

Example:

```
execution, err := registry.Execute(ctx, "shell_background",
    map[string]interface{}{
        "command": "npm run build",
    })
```

```
// Later, check status
status, err := registry.Execute(ctx, "shell_output",
    map[string]interface{}{
        "execution_id": execution.ID,
    })
```

shell_output

Get output from a background shell execution.

Parameters: - execution_id (string, required): ID from shell_background

Returns: ExecutionStatus with current output and state

shell_kill

Kill a running background shell execution.

Parameters: - `execution_id` (string, required): ID from `shell_background` -
`signal` (string, optional): Signal to send (SIGTERM, SIGKILL)

Returns: nil on success

Web Tools

`web_fetch`

Fetch content from a URL.

Parameters: - `url` (string, required): URL to fetch - `parse_markdown` (boolean, optional): Parse HTML to markdown (default: true) - `follow_redirects` (boolean, optional): Follow redirects (default: true)

Returns: Markdown content and metadata or raw `FetchResult`

Example:

```
markdown, metadata, err := registry.Execute(ctx, "web_fetch",
    map[string]interface{}{
        "url": "https://example.com/docs",
        "parse_markdown": true,
    })
```

Features: - Automatic HTML to Markdown conversion - Caching with TTL - Rate limiting - User-agent rotation - Redirect handling

Security: - Blocks private IP addresses - Domain blocklist (.onion, etc.) - Content-Type validation - Size limits (10 MB default)

`web_search`

Search the web for information.

Parameters: - `query` (string, required): Search query - `max_results` (integer, optional): Maximum results (default: 10) - `provider` (string, optional): Search provider (google, bing, duckduckgo)

Returns: `SearchResult` with URLs, titles, snippets

Example:

```
results, err := registry.Execute(ctx, "web_search",
    map[string]interface{}{
        "query": "golang best practices 2025",
    })
```

```
    "max_results": 5,  
  })
```

Providers: - DuckDuckGo (default, no API key required) - Google (requires API key and CSE ID) - Bing (requires API key)

Browser Tools

browser_launch

Launch a new browser instance.

Parameters: - headless (boolean, optional): Run in headless mode (default: true) - width (integer, optional): Window width (default: 1280) - height (integer, optional): Window height (default: 720)

Returns: Browser object with ID and WebSocket URL

Example:

```
browser, err := registry.Execute(ctx, "browser_launch",  
    map[string]interface{}{  
        "headless": true,  
        "width": 1920,  
        "height": 1080,  
    })
```

browser_navigate

Navigate browser to a URL.

Parameters: - browser_id (string, required): Browser instance ID - url (string, required): URL to navigate to

Returns: nil on success

browser_screenshot

Take a screenshot of the browser window.

Parameters: - browser_id (string, required): Browser instance ID - format (string, optional): Image format (png, jpeg) - quality (integer, optional): Image quality 0-100 - annotate (boolean, optional): Annotate interactive elements

Returns: Screenshot object with image data and metadata

Example:

```
screenshot, err := registry.Execute(ctx, "browser_screenshot",
    map[string]interface{}{
        "browser_id": browser.ID,
        "format": "png",
        "annotate": true,
    })
```

Annotation Features: - Labels interactive elements (buttons, links, inputs) - Generates element IDs for clicking/typing - Extracts bounding boxes - Highlights hover states

browser_close

Close a browser instance.

Parameters: - browser_id (string, required): Browser instance ID

Returns: nil on success

Mapping Tools

codebase_map

Create a map of the codebase structure and definitions.

Parameters: - root (string, optional): Root directory to map - languages (array, optional): Languages to include - use_cache (boolean, optional): Use cached results (default: true)

Returns: CodebaseMap with files, definitions, dependencies

Example:

```
codebaseMap, err := registry.Execute(ctx, "codebase_map",
    map[string]interface{}{
        "languages": []string{"go", "python", "typescript"},
        "use_cache": true,
    })
```

Extracted Information: - Functions, methods, classes, structs - Imports and dependencies - Comments and docstrings - Line counts and complexity metrics - Symbol locations (file, line, column)

Supported Languages: - Go, Python, JavaScript/TypeScript, Java, C/C++, Rust, Ruby, PHP

file_definitions

Get all definitions from a specific file.

Parameters: - path (string, required): Path to the file

Returns: FileMap with definitions, imports, exports

Example:

```
definitions, err := registry.Execute(ctx, "file_definitions",
    map[string]interface{}{
        "path": "internal/server/router.go",
    })
```

MultiEdit Tools

Multi-file editing uses a transactional approach to safely edit multiple files atomically.

Workflow

1. **Begin** - Start a transaction
2. **Add** - Add file edits to the transaction
3. **Preview** - Review changes and conflicts
4. **Commit** - Apply all changes atomically (or Rollback)

multiedit_begin

Begin a multi-file edit transaction.

Parameters: - description (string, required): Description of the edit operation - require_preview (boolean, optional): Require preview before commit (default: true)

Returns: EditTransaction object with ID

Example:

```
tx, err := registry.Execute(ctx, "multiedit_begin",
    map[string]interface{}{
        "description": "Refactor authentication logic",
        "require_preview": true,
    })
```

multiedit_add

Add a file edit to an open transaction.

Parameters: - transaction_id (string, required): Transaction ID - file_path (string, required): Path to the file - operation (string, required): Operation type (create, update, delete) - new_content (string, required for create/update): New file content

Returns: nil on success

Example:

```
err := registry.Execute(ctx, "multiedit_add",
    map[string]interface{}{
        "transaction_id": tx.ID,
        "file_path": "internal/auth/handler.go",
        "operation": "update",
        "new_content": updatedContent,
    })
```

multiedit_preview

Preview changes in a multi-file edit transaction.

Parameters: - transaction_id (string, required): Transaction ID

Returns: PreviewResult with diffs, conflicts, summary

Example:

```
preview, err := registry.Execute(ctx, "multiedit_preview",
    map[string]interface{}{
        "transaction_id": tx.ID,
    })
```

```
// Check for conflicts
if preview.Summary.HasConflicts {
```

```
    // Handle conflicts
}
```

multiedit_commit

Commit a multi-file edit transaction.

Parameters: - transaction_id (string, required): Transaction ID

Returns: nil on success

Example:

```
err := registry.Execute(ctx, "multiedit_commit",
    map[string]interface{}{
        "transaction_id": tx.ID,
    })
```

Safety Features: - Automatic backups before changes - Checksum verification to detect concurrent modifications - Atomic commit (all or nothing) - Automatic rollback on failure - Git integration for detecting uncommitted changes

Interactive Tools

ask_user

Ask the user a question and wait for their response.

Parameters: - question (string, required): Question to ask - options (array, optional): Predefined options - default (string, optional): Default answer - timeout (integer, optional): Timeout in seconds

Returns: UserResponse with answer

Example:

```
response, err := registry.Execute(ctx, "ask_user",
    map[string]interface{}{
        "question": "Which authentication method should we use?",
        "options": []string{"JWT", "OAuth", "Session"},
        "default": "JWT",
    })
```

task_tracker

Track and manage tasks during execution.

Parameters: - action (string, required): Action (create, update, list, get, complete) - task_id (string, required for update/get/complete): Task ID - title (string, required for create): Task title - description (string, optional): Task description - status (string, optional): Status (pending, in_progress, completed, failed) - progress (integer, optional): Progress percentage (0-100)

Returns: Task object or array of tasks

Example:

```
// Create task
task, err := registry.Execute(ctx, "task_tracker",
    map[string]interface{}{
        "action": "create",
        "title": "Run integration tests",
        "description": "Execute all integration tests and verify
            results",
    })

// Update progress
_, err = registry.Execute(ctx, "task_tracker",
    map[string]interface{}{
        "action": "update",
        "task_id": task.ID,
        "status": "in_progress",
        "progress": 50,
    })

// Complete task
_, err = registry.Execute(ctx, "task_tracker",
    map[string]interface{}{
        "action": "complete",
        "task_id": task.ID,
    })
```

Notebook Tools

notebook_read

Read and parse a Jupyter notebook (.ipynb) file.

Parameters: - path (string, required): Path to the notebook file - include_outputs (boolean, optional): Include cell outputs (default: true)

Returns: Notebook object with cells and metadata

Example:

```
notebook, err := registry.Execute(ctx, "notebook_read",
    map[string]interface{}{
        "path": "analysis.ipynb",
        "include_outputs": true,
    })
```

notebook_edit

Edit a cell in a Jupyter notebook.

Parameters: - path (string, required): Path to the notebook file - cell_index (integer, optional): Cell index (0-based) - cell_id (string, optional): Cell ID (alternative to index) - source (string, required): New source code/markdown - cell_type (string, optional): Cell type (code, markdown) - operation (string, optional): Operation (replace, insert, delete)

Returns: Updated Notebook object

Example:

```
// Replace cell content
_, err := registry.Execute(ctx, "notebook_edit",
    map[string]interface{}{
        "path": "analysis.ipynb",
        "cell_index": 0,
        "source": "import pandas as pd\nimport numpy as np",
        "operation": "replace",
    })

// Insert new cell
_, err = registry.Execute(ctx, "notebook_edit",
    map[string]interface{}{
        "path": "analysis.ipynb",
        "cell_index": 1,
        "source": "# Data Processing\nThis section handles data\n    processing.",
        "cell_type": "markdown",
        "operation": "insert",
    })
```

Security Considerations

Path Validation

All file operations validate paths to prevent: - Directory traversal attacks (`../../../../etc/passwd`) - Access outside workspace boundaries - Symlink exploitation - Access to sensitive files (`.env`, credentials, keys)

Command Execution

Shell commands are protected by: - Blocklist of dangerous commands (`rm -rf /`, `dd`, `mkfs`, etc.) - Resource limits (memory, CPU, processes) - Timeout enforcement - Environment isolation - Audit logging

Web Access

Web operations implement: - Domain blocklist (`.onion`, private IPs, etc.) - Rate limiting - Content-Type validation - Size limits - User-agent rotation - SSL/TLS verification

Multi-Edit Safety

Transactional edits ensure: - Atomic operations (all or nothing) - Backup creation before changes - Checksum verification - Conflict detection - Automatic rollback on failure

Best Practices

Error Handling

Always check errors and handle them appropriately:

```
result, err := registry.Execute(ctx, "fs_read", params)
if err != nil {
    // Check for specific error types
    if errors.Is(err, filesystem.ErrFileNotFound) {
        // Handle missing file
    } else if errors.Is(err, filesystem.ErrPermissionDenied) {
        // Handle permission error
    } else {
        // Handle other errors
    }
}
```

```
    return err
}
```

Context Management

Use contexts for cancellation and timeouts:

```
ctx, cancel := context.WithTimeout(context.Background(),
    30*time.Second)
defer cancel()

result, err := registry.Execute(ctx, "web_fetch", params)
```

Resource Cleanup

Always close the registry when done:

```
registry, err := tools.NewToolRegistry(config)
if err != nil {
    return err
}
defer registry.Close()
```

Validation

Validate parameters before execution:

```
tool, err := registry.Get("fs_read")
if err != nil {
    return err
}

if err := tool.Validate(params); err != nil {
    // Handle validation error
    return err
}

result, err := tool.Execute(ctx, params)
```

Performance

For repeated operations, reuse the registry:

```
// Bad: Creating new registry for each operation
for _, file := range files {
    registry, _ := tools.NewToolRegistry(config)
```

```

    registry.Execute(ctx, "fs_read", map[string]interface{}{
        "path": file})
    registry.Close()
}

// Good: Reuse registry
registry, err := tools.NewToolRegistry(config)
defer registry.Close()

for _, file := range files {
    registry.Execute(ctx, "fs_read", map[string]interface{}{
        "path": file})
}

```

Configuration

Registry Configuration

Customize the registry behavior:

```

config := tools.DefaultRegistryConfig()

// FileSystem configuration
config.FileSystemConfig.MaxFileSize = 100 * 1024 * 1024 // 100 MB
config.FileSystemConfig.CacheEnabled = true
config.FileSystemConfig.CacheTTL = 5 * time.Minute
config.FileSystemConfig.BlockedPaths = []string{".git",
    "node_modules", ".env"}

// Shell configuration
config.ShellConfig.MaxConcurrent = 10
config.ShellConfig.DefaultTimeout = 30 * time.Second
config.ShellConfig.MaxTimeout = 10 * time.Minute
config.ShellConfig.AuditLog = true

// Web configuration
config.WebConfig.CacheEnabled = true
config.WebConfig.RateLimitEnabled = true
config.WebConfig.MaxContentSize = 10 * 1024 * 1024 // 10 MB

// Browser configuration
config.BrowserConfig.MaxConcurrentBrowsers = 5
config.BrowserConfig.DefaultHeadless = true

```

```
registry, err := tools.NewToolRegistry(config)
```

Extending the Tool System

Creating Custom Tools

Implement the Tool interface:

```
type CustomTool struct {
    registry *ToolRegistry
}

func (t *CustomTool) Name() string {
    return "custom_tool"
}

func (t *CustomTool) Description() string {
    return "Description of what the tool does"
}

func (t *CustomTool) Category() ToolCategory {
    return CategoryFileSystem
}

func (t *CustomTool) Schema() ToolSchema {
    return ToolSchema{
        Type: "object",
        Properties: map[string]interface{}{
            "param1": map[string]interface{}{
                "type": "string",
                "description": "Parameter description",
            },
        },
        Required: []string{"param1"},
        Description: "Tool description",
    }
}

func (t *CustomTool) Validate(params map[string]interface{}) error {
    if _, ok := params["param1"]; !ok {
        return fmt.Errorf("param1 is required")
    }
}
```

```

    }
    return nil
}

func (t *CustomTool) Execute(ctx context.Context, params
    map[string]interface{}) (interface{}, error) {
    // Implementation
    return result, nil
}

// Register the tool
registry.Register(&CustomTool{registry: registry})

```

Examples

Example 1: Analyze and Refactor Code

```

ctx := context.Background()
registry, _ :=
    tools.NewToolRegistry(tools.DefaultRegistryConfig())
defer registry.Close()

// Map the codebase
codebaseMap, _ := registry.Execute(ctx, "codebase_map",
    map[string]interface{}{
        "languages": []string{"go"},
    })

// Find all TODO comments
todos, _ := registry.Execute(ctx, "grep", map[string]interface{}{
    "pattern": "TODO:",
    "case_sensitive": false,
})

// Start multi-file edit
tx, _ := registry.Execute(ctx, "multiedit_begin",
    map[string]interface{}{
        "description": "Address TODO comments",
    })

// Add edits for each TODO
for _, todo := range todos.([]ContentMatch) {
    // Process TODO and add edit

```

```

registry.Execute(ctx, "multiedit_add", map[string]interface{}{
    {
        "transaction_id": tx.ID,
        "file_path": todo.Path,
        "operation": "update",
        "new_content": processedContent,
    })
}

// Preview changes
preview, _ := registry.Execute(ctx, "multiedit_preview",
    map[string]interface{}{
        "transaction_id": tx.ID,
    })

// Commit if no conflicts
if !preview.Summary.HasConflicts {
    registry.Execute(ctx, "multiedit_commit",
        map[string]interface{}{
            "transaction_id": tx.ID,
        })
}

```

Example 2: Web Research and Documentation

```

// Search for documentation
results, _ := registry.Execute(ctx, "web_search",
    map[string]interface{}{
        "query": "Go context best practices",
        "max_results": 5,
    })

// Fetch and parse top result
markdown, metadata, _ := registry.Execute(ctx, "web_fetch",
    map[string]interface{}{
        "url": results[0].URL,
        "parse_markdown": true,
    })

// Create documentation file
registry.Execute(ctx, "fs_write", map[string]interface{}{
    "path": "docs/context-best-practices.md",
    "content": markdown,
})

```

Example 3: Automated Testing with Browser

```
// Launch browser
browser, _ := registry.Execute(ctx, "browser_launch",
    map[string]interface{}{
        "headless": true,
    })

// Navigate to application
registry.Execute(ctx, "browser_navigate", map[string]interface{}{
    "browser_id": browser.ID,
    "url": "http://localhost:3000",
})

// Take screenshot
screenshot, _ := registry.Execute(ctx, "browser_screenshot",
    map[string]interface{}{
        "browser_id": browser.ID,
        "annotate": true,
    })

// Save screenshot
registry.Execute(ctx, "fs_write", map[string]interface{}{
    "path": "screenshots/homepage.png",
    "content": screenshot.Data,
})

// Close browser
registry.Execute(ctx, "browser_close", map[string]interface{}{
    "browser_id": browser.ID,
})
```

Troubleshooting

Common Issues

Issue: “path is outside workspace” - **Solution:** Ensure all file paths are within the configured workspace root

Issue: “command blocked by security policy” - **Solution:** Review the command for dangerous operations or use allowlist mode

Issue: “too many concurrent browsers” - **Solution:** Close unused browsers or increase MaxConcurrentBrowsers in config

Issue: “file was modified since transaction started” - **Solution:** Resolve conflicts manually or retry the transaction

Debug Mode

Enable detailed logging:

```
config := tools.DefaultRegistryConfig()
config.FileSystemConfig.CacheEnabled =
    false // Disable cache for debugging
config.ShellConfig.AuditLog = true // Enable audit log
```

```
registry, _ := tools.NewToolRegistry(config)
```

Performance Profiling

```
import "net/http/pprof"

go func() {
    http.ListenAndServe("localhost:6060", nil)
}()

// Access profiling at http://localhost:6060/debug/pprof/
```

API Reference

For detailed API documentation, see: - [GoDoc](#) - [OpenAPI Schema](#) (generated via `registry.ExportSchemas()`)

License

Copyright (c) 2025 HelixCode Project